

AN ILLUSTRATED C JOURNEY

The Whole Point of Pointers

*Addresses, aliases, arrays, dynamic memory -
and why C trusts you with all of it*



Lijo M. Jose

Assistant Professor, School of Computer Science and Engineering

Before You Press Start

This chapter unfolds inside a small retro game. Its main character is **Caitlyn**, a brave young knight entering a dungeon where the code does not always behave as the player expects. A monster attacks, but her health refuses to change. Her inventory outgrows its shelves. Later, even the targeting reticle must learn to point somewhere new.

These game situations are not decoration. Each one is designed to create a real programming need before its concept is named. We will first stay with Caitlyn inside the problem, predict what the code will do, and ask what kind of solution is missing. Only after that need becomes visible will the definition, memory picture, and C syntax enter the story.

Caitlyn's journey is our travelling companion, but the ideas belong far beyond games. Health bars, inventory racks, dungeon rooms, and targeting reticles simply make the problems easier to picture. The hope is that you finish each scene knowing not only **how** to write the code, but **when** the idea is useful, **where** it may appear again, and **why** it exists in the first place. That understanding is far more durable than memorising definitions and syntax.

How to travel through this chapter

- **Pause before the reveal.** When the story presents a bug or a design problem, predict what the program will do and what kind of solution may be needed.
- **Trace before you run.** Follow the variables, addresses, and arrows by hand. Then run the program and compare the machine's answer with yours.
- **Experiment after the explanation.** Change a value, a size, or one line of code. A small experiment often teaches more than rereading the paragraph.
- **Leave the dungeon for a moment.** For every concept, think of at least one situation outside this game where the same need could arise.

YOUR SIDE QUEST At the end of each discovery, ask: **Where else would this be useful? What difficulty would appear without it? Could another approach solve the same problem, and what would it cost?** Write down one answer of your own before moving on.

Before the dungeon door opens

Five doors wait ahead. We will not open any of them until the game gives us a reason.

Stage	Question unlocked	Ideas earned
ACT I	The hero will not take damage	values, addresses, dereferencing, aliases
ACT II	The inventory becomes a corridor	arrays, pointer movement, strings, racks
ACT III	The quartermaster needs more space	allocation, resizing, lifetime, ownership
ACT IV	The targeting route must move	pointer-to-pointer and output parameters
ACT V	The dungeon stops forgiving mistakes	bounds, dangling routes, cleanup, tools
CORE IDEA We begin with one ordinary integer. Every stranger-looking piece of syntax must earn its entrance by solving a problem the hero has already met.		

Quest markers

MAIN QUEST - master this path on the first reading; later ideas depend on it.

SIDE QUEST - a useful extension; explore it after the main route feels steady.

BOSS DOOR - advanced and optional on the first pass. Return when you want the extra safety or flexibility.

Reach the Original Object



Values, addresses, dereferencing, and aliases

Mission: help a function reach the hero's real health - not merely a copy of it.

1. The Hero Who Refused to Take Damage

The dungeon was ready.

Torches flickered along the stone walls. A monster waited behind the first door. At the top of the screen, our hero Caitlyn had a bright green health bar and exactly 100 health points.

The monster attacked.

Twenty points of damage flashed above Caitlyn's head. The sound effect played. The screen shook with great enthusiasm.

And the health bar remained at 100.

This was not courage. It was a bug.

The damage function looked innocent enough:

```
C
#include <stdio.h>

void takeDamage(int health)
{
    health = health - 20;
}

int main(void)
{
    int heroHealth = 100;

    takeDamage(heroHealth);

    printf("Hero health: %d\n", heroHealth);
    return 0;
}
```

PAUSE AND PREDICT The monster has attacked. Will the program print 80 or 100? Do not ask the compiler yet. Run the scene in your head.

The program prints:

```
TEXT
Hero health: 100
```

Caitlyn has discovered the least useful kind of immortality: the kind caused by a misunderstanding of function calls.

Let us replay the moment slowly.

Inside **main**, the variable **heroHealth** contains 100. When C reaches this line:

```
C
takeDamage(heroHealth);
```

it does not send the variable itself into the function. It reads the value stored there - 100 - and uses that value to create a new local variable named **health**.

For a brief moment, two different integers exist:

```
TEXT
main()                                takeDamage()
heroHealth = 100    --copy-->    health = 100
```

The function subtracts 20 from its own **health**. That copy becomes 80. The original **heroHealth** remains peacefully untouched in **main**.

Moment	main()	takeDamage()
Before call	heroHealth = 100	-
Inside function	heroHealth = 100	health = 80
After return	heroHealth = 100	health no longer exists

Figure 1. The function changes its copy, not the caller's integer.

Then **takeDamage** ends. Its local variable disappears with the rest of the function call. The only 80 in the program vanishes, and the hero walks away with the original 100.

This is not a strange rule invented only for our unlucky hero. C passes arguments **by value**. A parameter is a fresh local object initialized from the value supplied by the caller.

A language note before we continue: in C, an object is simply a region of storage that holds a value. An int, an array, a structure, and a dynamically allocated block are all C objects. It does not necessarily mean an instance of a class as it usually does in OOP.

So the damage code did exactly what we told it to do. It damaged a copy.

The real question is no longer, "How do we subtract 20?" We already know how to do that.

The real question is:

CORE IDEA How can **takeDamage** reach the actual **heroHealth** living inside **main**?

That question opens the first door.

2. Give the Function Directions

Imagine Caitlyn's health is kept in one particular room inside the game's memory. The number 100 is inside that room. Passing **heroHealth** gives the function a copy of what is inside.

But what if we give the function the room number instead?

Every object in memory has a location. C calls that location its **address**, and the operator **&** asks for it:

```
C
&heroHealth
```

Read that expression as:

CORE IDEA the address of **heroHealth**

We are no longer handing **takeDamage** another copy of 100. We are handing it directions to the place where the real 100 lives:

```
C
takeDamage(&heroHealth);
```

Of course, the function must now be prepared to receive an address. An ordinary **int** stores an integer. We need a variable that can store the address of an integer.

That variable is a pointer:

```
C
void takeDamage(int *health)
{
    *health = *health - 20;
}
```

The complete game now looks like this:

```
C
#include <stdio.h>

void takeDamage(int *health)
{
    *health = *health - 20;
}

int main(void)
{
    int heroHealth = 100;

    takeDamage(&heroHealth);

    printf("Hero health: %d\n", heroHealth);
    return 0;
}
```


This time:

```
TEXT
Hero health: 80
```

At last, the monster is dangerous. Game balance has been restored.

But two new symbols have entered the story, and they deserve more than hurried definitions.

Suppose we write:

```
C
int heroHealth = 100;
int *healthPtr = &heroHealth;
```

There is still only one health integer. **healthPtr** does not contain another 100. It contains the address of the original one.

Expression	Produces	Meaning
heroHealth	100	The int object
&heroHealth	0x5000	The int object's address (symbolic)
healthPtr	0x5000	A pointer storing that address
*healthPtr	100	The int reached through the pointer
&healthPtr	0x7000	The pointer variable's own address

Figure 2. One integer, one pointer, and two distinct storage locations.

Now read each expression as a small action.

heroHealth means, "Use the integer directly." In most expressions, that produces 100.

&heroHealth means, "Tell me where that integer lives."

healthPtr means, "Use the address stored in this pointer variable."

And ***healthPtr** means, "Follow that stored address and use the integer found there."

That last operation is called **dereferencing**. The name sounds ceremonial; the action is simple. Follow the directions. Reach the object.

```
C
printf("%d\n", *healthPtr); // reach heroHealth and read 100

*healthPtr = 70;           // reach heroHealth and change it
```

After the assignment, **heroHealth** is 70. We have not created a second health value. We have created a second route to the same health value.

The direct route is the name **heroHealth**. The indirect route is **healthPtr**. Two routes to one object are called **aliases**.

CORE IDEA A pointer is not the object. It is a stored route to the object.

Why does the route have a type?

The game engine is now holding an address. But an address alone does not say what waits at the destination.

Is it one **int**? A **char**? A **double**? An entire enemy?

That is why the pointed-to type appears in the declaration:

```
C
int  *healthPtr;
char *symbolPtr;
double *distancePtr;
```

When the compiler sees **int *healthPtr**, it learns that following **healthPtr** should lead to an **int**. That knowledge tells it how many bytes form one object there, how to interpret those bytes, which operations make sense, and - later - how far one step of pointer movement should travel.

Read the declaration from the variable name outward:

CORE IDEA **healthPtr** is a pointer to an **int**.

One small declaration can still set a trap:

```
C
int *p, q;
```

Only **p** is a pointer. **q** is an ordinary **int**. The ***** belongs to each individual declarator, not to the word **int** floating on the left. Declaring one variable per line is not mandatory, but it keeps this particular goblin out of the code.

The copied directions

Now pause. Did passing an address somehow turn C into a pass-by-reference language?

No. C is still copying a value. The copied value just happens to be an address.

Suppose **heroHealth** lives at the imaginary address 1000:

TEXT

```
heroHealth  
address: 1000  
value: 100
```

The call:

C

```
takeDamage(&heroHealth);
```

produces the value 1000 and copies it into the parameter **health**. The parameter is a different variable, with its own storage and its own address.

Variable	Its own address	Value stored inside	What it is
heroHealth	1000	100	The original integer in main()
health	2000	1000	A local pointer in takeDamage()

Figure 3. Two separate variables; the value inside health leads to heroHealth.

health stores 1000 -> address 1000 contains heroHealth

There are now two variables doing two different jobs:

- **heroHealth** is the original integer. It contains 100.
- **health** is a local pointer. It contains 1000, the address of **heroHealth**.

The two variables are not the same. Yet the value stored in **health** is a route back to **heroHealth**.

So this line:

C

```
*health = *health - 20;
```

does not change the pointer variable. It follows the address inside the pointer and changes the integer waiting there.

Now compare it with:

C

```
health = NULL;
```

This line changes the local pointer **health** itself. It throws away the copied route. It does not change **heroHealth**, and it would not change a pointer variable belonging to the caller.

The distinction is worth holding onto:

CORE IDEA Changing ***health** changes the object at the destination. Changing **health** changes only the local copy of the route.

Much later, the game's targeting system will need to change a route owned by its caller. When that moment arrives, one pointer will not be enough. But the targeting room is still several doors away.

3. Could We Simply Return the New Value?

At this point, a sensible programmer may lean back and say:

"This is a lot of ceremony for subtracting 20. Why not return the new health?"

Fair objection.

```
C
int takeDamage(int health)
{
    return health - 20;
}

int main(void)
{
    int heroHealth = 100;
    heroHealth = takeDamage(heroHealth);
}
```

For this tiny scene, that solution is perfectly good. It may even be clearer. We should not invent a fake need for pointers merely because the chapter title has already promised them.

Then the second monster enters.

This one does not merely reduce health. Its attack cracks the shield, awards survival points, and may leave the hero alive or defeated. One battle changes several pieces of the caller's game state.

Which one should the function return?

We could return health and forget the score. Return the score and forget the shield. Call three separate functions and hope the battle rules do not drift apart. Use global variables and invite every function in the program to touch them.

Or we can give one battle function routes to all three original objects:

```
C
void resolveBattle(int *health, int *shield, int *score)
{
    int attack = 25;

    if (*shield >= attack) {
        *shield -= attack;
    } else {
        attack -= *shield;
        *shield = 0;
        *health -= attack;
    }

    *score += 50;
}
```

The caller supplies the addresses:

```
C
int heroHealth = 100;
int heroShield = 10;
int heroScore = 200;

resolveBattle(&heroHealth, &heroShield, &heroScore);
```

When the dust settles:

```
TEXT
health = 85
shield = 0
score = 250
```

The function still directly returns only one value - in this case, nothing, because its return type is **void**. Through pointers, however, it reaches and modifies several caller-owned objects.

As the hero grows, those three values begin to feel like parts of one thing rather than unrelated variables. That suggests a structure:

A quick structure checkpoint prevents four new symbols from arriving at once. typedef gives the structure type the short name Hero. A Hero variable uses . to reach a field. A pointer to Hero uses -> because the program must first follow the pointer.

```
C
typedef struct {
    int health;
    int shield;
    int score;
} Hero;

void resolveBattle(Hero *hero)
{
    hero->health -= 15;
    hero->shield = 0;
    hero->score += 50;
}

int main(void)
{
    Hero caitlyn = {100, 10, 200};
    caitlyn.health = 95;    // object: use .

    Hero *heroPtr = &caitlyn;
    heroPtr->health = 85;    // pointer: use ->
    (*heroPtr).health = 80;    // exactly the same field access

    resolveBattle(heroPtr);
    return 0;
}
```

The expression `heroPtr->health` is shorthand for `(*heroPtr).health`. The parentheses matter because `.` binds before `*`. The arrow simply writes this common dereference-and-select operation more cleanly.

The pointer idea has not disappeared. It has become cleaner. Instead of passing three routes, we pass the address of one **Hero** object and update its fields through that route.

The first lesson was not "always use pointers." It was more useful:

CORE IDEA Use a pointer when a function must reach an existing object rather than work only with a copied value.

4. Safe First Steps with Pointer Variables

The game now has a working health system. Caitlyn is ready to save the progress.

The programmer creates a pointer and immediately writes through it:

```
C
int *healthPtr;
*healthPtr = 100;
```

There is one problem: we created a signpost but never wrote a destination on it.

healthPtr is uninitialized. Its bits contain no meaningful address deliberately supplied by our program. Dereferencing it tells C, "Go to whatever location these leftover bits happen to describe and write 100 there."

Perhaps that location is invalid and the program crashes. Perhaps it belongs to another object and the program quietly corrupts it. Perhaps the demonstration appears to work once and fails in front of the class. C offers all three genres.

If a pointer does not yet have a destination, say so explicitly:

```
C
int *healthPtr = NULL;
```

NULL means "points to no object." It is not a secret emergency object and it must not be dereferenced. But it can be tested:

```
C
if (healthPtr != NULL) {
    printf("%d\n", *healthPtr);
}
```

That turns an accidental unknown route into a deliberate empty route.

Sometimes we want to see a pointer value while debugging. **%p** is the format for printing an address, and the argument should be converted to **void ***:

```
C
printf("%p\n", (void *)healthPtr);
```

The result often looks hexadecimal. Treat it as a clue for this execution, not as a permanent room number. The operating system and runtime may place the same object somewhere else next time.

Before we leave this room, put three expressions side by side:

```
C
p    // the address stored in p
*p   // the object reached through that address
p + 1 // the next object-sized position
```

The third expression has not yet earned a full explanation. A pointer should not wander through unrelated memory. But just beyond the next door, the hero's inventory is arranged in a perfect line, and one-object steps are about to become extremely useful.

SAVE POINT · ACT I REPLAY

Do not rush through the portal. Close the chapter for a moment, make your prediction, and draw the routes before running the code.

```
C
void forgetRoute(int *p)
{
    p = NULL;
}

int health = 100;
int *target = &health;
forgetRoute(target);
printf("%d %d\n", health, target == NULL);
```

PREDICT · What two values are printed? Explain why changing `p` does not change `target`.

DRAW THE ROUTES · Draw `health`, `target`, and the parameter `p` as three separate objects. Which two contain the same address during the call?

CHANGE THE GAME · Rewrite `forgetRoute` so it changes the caller's integer `health` to 0 instead of changing the copied pointer.

LEAVE THE DUNGEON · Name one non-game situation where a function must update several caller-owned values.

You are ready to continue when you can explain the result without using the phrase “because pointers do that.”

Walk Through Ordered Memory



Arrays, decay, pointer arithmetic, strings, and two-dimensional rows

Mission: discover how one address can become a route through an entire inventory.

5. Arrays Were Quietly Doing This All Along



Caitlyn reaches one item in a contiguous inventory.

The quartermaster opens Caitlyn's inventory. Five slots glow in a row:

```
C
int inventory[5] = {40, 12, 99, 7, 25};
```

This is not merely a convenient drawing. The five integers occupy one contiguous sequence of memory. Slot 1 follows slot 0. Slot 2 follows slot 1. No unrelated object is inserted between them.

Now something that C has been doing quietly becomes visible. In most expressions, the array name **inventory** is converted to a pointer to its first element:

```
C
int *p = inventory;
```

The pointer **p** now stores the same starting address as **&inventory[0]**.

p -> &inventory[0]

inventory[0]	inventory[1]	inventory[2]	inventory[3]	inventory[4]
40	12	99	7	25
0x8000	0x8004	0x8008	0x800C	0x8010

Figure 4. Contiguous int elements; symbolic addresses assume a 4-byte int.

This automatic conversion is commonly called **array decay**. The array itself has not transformed into a pointer. The expression has produced a pointer to the first element so that code can reach the sequence.

The quartermaster asks for slot 2. We can reach it in several ways:

```
C
inventory[2]
*(inventory + 2)
p[2]
*(p + 2)
```

All four expressions lead to the same **int**.

That is because C defines array indexing through pointer arithmetic:

```
C
inventory[i] == *(inventory + i)
p[i]         == *(p + i)
```

When **p** is an **int ***, **p + 1** does not move one byte. It moves by the size of one **int**, landing at the next inventory slot. The pointed-to type decides the step.

The invisible finish line

The hero can walk from the first slot to the last:

```
C
int *begin = inventory;
int *end   = inventory + 5;
```

begin points to slot 0. **inventory + 4** points to slot 4, the final real element.

Then why is **inventory + 5** allowed?

Because loops need a clean finish line. C permits a pointer to be formed **one position beyond** the final element. That pointer does not point to an object, so it must not be dereferenced. It exists only as a boundary that can be compared against.

```
C
for (int *current = begin; current < end; current++)
    printf("%d ", *current);
```

The loop dereferences **current** only while it points to one of the five real slots. After printing slot 4, **current** advances to **end**. The condition becomes false before ***current** is evaluated again.

Think of **end** as the exit sign painted just beyond the corridor. You may walk up to it. You may compare your position with it. There is no inventory slot inside it.

Going farther has no such permission:

```
C
int *lost = inventory + 6; // outside the allowed range
```

Pointer arithmetic is meaningful only inside this array, plus its one-past boundary. Even if another array happens to be placed nearby, walking beyond **inventory** does not legally enter it. The pointer is a route through one corridor, not a licence to explore the entire building.

Is the array itself a pointer?

At the entrance, **inventory** and **p** cooperate so smoothly that it is tempting to say, "An array is a pointer."

Then the illusion cracks:

```
C
sizeof inventory // size of all five integers
sizeof p         // size of one pointer
```

And again:

```
C
p++;           // valid: p is a modifiable pointer variable
inventory++;   // invalid: inventory is an array name
```

The array owns storage for five integers. The pointer owns storage for one address. An array expression often gives us a pointer to its first element, but that does not make the two objects identical.

There are contexts where the array does not decay. Two of the most important are **sizeof** and unary **&**:

```
C
sizeof inventory
&inventory
```

The expression **&inventory** means "the address of the whole array." At first this seems pointless. Does not **inventory** already produce the same starting address?

Numerically, the two addresses often look the same. Their types tell different stories:

```
C
int *first = inventory; // points to one int
int (*whole)[5] = &inventory; // points to one array of five int
```

first + 1 moves to the next integer. **whole + 1** moves past one entire five-integer array.

The address is the same doorway; the type decides whether one step means "next slot" or "next complete five-slot inventory."

Read the second declaration from **whole** outward. Parentheses make **whole** a pointer. Dereferencing it produces an array of five integers:

CORE IDEA **whole** is a pointer to an array of five **int**.

This idea will return when one step must move across an entire rack rather than one slot.

Sending the inventory to a function

The quartermaster writes a small upgrade function:

```
C
void addPotion(int inventory[], int count)
{
    for (int i = 0; i < count; i++)
        inventory[i] += 1;
}
```

Inside a function parameter list, **int inventory[]** is adjusted to **int *inventory**. These two declarations describe the same parameter type:

```
C
void addPotion(int inventory[], int count);
void addPotion(int *inventory, int count);
```

The function does not receive a copy of all five elements. It receives a copied address leading to the first element. Through that address it changes the caller's original slots.

So this is still pass-by-value: the address is copied. The array contents are shared through the copied route.

There is one price. A pointer to the first element does not carry the array length. Inside the function, **sizeof inventory** measures the pointer, not the caller's five-element array. That is why **count** travels beside it.

PAUSE AND PREDICT If the caller passes **int a[5]** to **void f(int a[])**, can **f** recover 5 using **sizeof a / sizeof a[0]**? No. By the time the parameter exists, it is a pointer and the original array-size information is gone.

SIDE QUEST: The Hero Needs a Name

The game now asks the player to name the hero:

```
C
char heroName[] = "Caitlyn";
```

A C string is a character sequence ending with the null character `\0`. The literal **"Caitlyn"** therefore supplies eight characters:

```
TEXT
'C' 'a' 'i' 't' 'l' 'y' 'n' '\0'
```

Something special happens here. Because the string literal is initializing an array, C creates a new writable array named **heroName** and copies those eight characters into it. The literal is being used as the array's initial contents; this is not the ordinary array-to-pointer conversion.

That is why this change is valid:

```
C
heroName[0] = 'K'; // heroName now contains "Kaitlyn"
```

Now compare it with:

```
C
const char *title = "Hero";
```

Here **title** is a pointer referring to the string literal. It does not own a new writable character array. String literals must not be modified, so **const char *** states the intended read-only use.

Once the name lives in a character array, a pointer can walk through it:

```
C
char *cursor = heroName;

while (*cursor != '\0') {
    putchar(*cursor);
    cursor++;
}
```

The null character is the finish line. Functions such as **printf("%s", heroName)** also start from the first character and continue until they meet `\0`.

The name with a space in it

Caitlyn is a perfectly respectable name. The player, naturally, types something grander:

```
TEXT
Caitlyn the Unreasonably Brave
```

The first attempt uses **scanf**:

```
C
char heroName[50];
scanf("%49s", heroName);
```


The width **49** is important. It leaves one array position for the terminating **\0**, preventing this particular call from overflowing the 50-character storage.

But **%s** stops at the first whitespace. The game welcomes only **Caitlyn**. The rest of the title remains waiting in the input stream, presumably offended.

For a whole line, the game needs **fgets**:

```
C
fgets(heroName, sizeof heroName, stdin);
```

fgets knows the capacity of the destination and can read spaces. If the Enter newline fits, it stores that too. We can remove it with **strcspn**:

```
C
#include <stdio.h>
#include <string.h>

int main(void)
{
    char heroName[50];

    printf("Name your hero: ");

    if (fgets(heroName, sizeof heroName, stdin) == NULL)
        return 1;

    heroName[strcspn(heroName, "\n")] = '\0';
    printf("Welcome, %s!\n", heroName);

    return 0;
}
```

Question	scanf("%s", ...)	fgets(...)
Stops at	First whitespace	Newline or full capacity
Reads spaces	No, with %s	Yes
Capacity control	Use a width such as %49s	Pass the array size
Newline handling	Usually leaves it unread	Stores it when it fits
Good fit	One whitespace-free token	Names, sentences, commands

Table 1. Both functions receive a pointer to writable storage, but their input rules differ.

For one whitespace-free token, a width-limited **scanf("%49s", heroName)** may be enough. For a name, sentence, or command that can contain spaces, **fgets** usually matches the problem better.

Mixing the two requires care. **scanf** often leaves the Enter newline behind; the next **fgets** may consume that newline and appear to read an empty line.

Neither input function creates storage. Both receive a pointer to storage that must already exist. A pointer is still only the route. The writable destination and its capacity remain our responsibility.

6. A Two-Dimensional Array Is an Array of Arrays

The collector returns with more loot than five slots can hold.

"One corridor is no longer enough," says the quartermaster. "We need racks. Three racks. Four slots on each."

The declaration is:

```
C
int inventory[3][4];
```

Do not picture an **int ****. Do not imagine three hidden row pointers. Nothing in this declaration creates pointer objects.

Begin with the outermost object. **inventory** is an array with three elements. What is each element? An array of four integers.

So the memory is arranged as three adjacent rack objects:

```
TEXT
inventory[0] : four adjacent int slots
inventory[1] : four adjacent int slots
inventory[2] : four adjacent int slots
```

And inside each rack:

```
TEXT
inventory[1][0] inventory[1][1] inventory[1][2] inventory[1][3]
```

All twelve integers are contiguous in **row-major order**. The four slots of rack 0 come first, followed immediately by the four slots of rack 1, then rack 2.

	column 0	column 1	column 2	column 3
row 0	inventory[0][0]	inventory[0][1]	inventory[0][2]	inventory[0][3]
row 1	inventory[1][0]	inventory[1][1]	inventory[1][2]	inventory[1][3]
row 2	inventory[2][0]	inventory[2][1]	inventory[2][2]	inventory[2][3]

Figure 5. Three inventory racks stored side by side; each rack contains four adjacent slots.

Now ask the question that controls everything:

CORE IDEA What is one element of **inventory**?

It is not one integer. It is one complete rack - an array of four integers.

Therefore, when **inventory** decays in an expression, it becomes a pointer to its first rack. The matching type is:

```
C
int (*rackPtr)[4] = inventory;
```

Read it from **rackPtr** outward.

The parentheses make **rackPtr** a pointer. Dereferencing it gives an array of four elements. Those elements are **int** values.

CORE IDEA **rackPtr** is a pointer to an array of four **int**.

Without the parentheses:

```
C
int *rackPtr[4];
```

the brackets bind first. That declaration creates an array of four pointers to **int** - a completely different object.

When one step means one rack

Because **rackPtr** points to a whole rack, moving it by one skips four integers:

```
C
rackPtr    // points to inventory[0]
rackPtr + 1 // points to inventory[1]
rackPtr + 2 // points to inventory[2]
```

Dereferencing **rackPtr** produces the first rack array:

```
C
*rackPtr    // inventory[0]
(*rackPtr)[2] // inventory[0][2]
```

Several expressions may begin at the same numerical address while describing different-sized steps:

```
C
inventory    // pointer to a rack: int (*)[4]
inventory[0] // decays to pointer to int: int *
&inventory[0][0] // pointer to the first int: int *

inventory + 1 // next rack: skips four int objects
inventory[0] + 1 // next slot: skips one int object
```

The starting doorway is shared. The type tells C whether one step crosses a slot or an entire rack.

Opening one exact slot

What really happens when the program writes:

```
C
inventory[i][j]
```

The first indexing operation chooses a rack. The second chooses a slot inside that rack.

Because indexing is defined through pointer arithmetic:

```
C
inventory[i]    == *(inventory + i)
inventory[i][j] == *(* (inventory + i) + j)
```

Follow the route slowly:

1. **inventory + i** moves to rack **i**.
2. *** (inventory + i)** reaches that rack array.
3. In the next expression, the rack array decays to a pointer to its first **int**.
4. Adding **j** moves to slot **j**.
5. The outer ***** reaches the integer stored there.

Trace one real value

```
C
int inventory[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};
```

Now evaluate `* (* (inventory + 1) + 2)` without skipping a step:

```
TEXT
inventory + 1    -> address of row 1
*(inventory + 1)  -> row 1: {40, 50, 60}
*(inventory + 1) + 2 -> address of row 1, column 2
*(* (inventory + 1) + 2) -> 60
```

The first movement chooses a row-sized object; the second movement chooses one int inside that row. The two additions use different step sizes because their pointer types are different.

This is why the compiler must know how many columns - how many slots per rack - exist. Without the rack width, it cannot know how far **inventory + 1** should move.

Letting a function enter the racks

If every inventory always has four slots per rack, a function can declare:

```
C
void printFixedInventory(int racks, int inventory[][4]);
```

In the parameter list, `inventory[][4]` is adjusted to `int (*inventory)[4]`: a pointer to the first four-slot rack. The number of racks still travels separately.

C99 - the 1999 revision of the C language - also allows the rack width to arrive at runtime. The dimensions must appear before the array parameter so the compiler knows the row type when it reaches that declaration:

```
C
void printInventory(int racks, int slotsPerRack,
                  int inventory[racks][slotsPerRack])
{
    for (int rack = 0; rack < racks; rack++) {
        for (int slot = 0; slot < slotsPerRack; slot++)
            printf("%d ", inventory[rack][slot]);
        putchar('\n');
    }
}
```

Inside the function, the parameter behaves conceptually as a pointer to a runtime-sized rack: `int (*inventory)[slotsPerRack]`.

Here is the full fixed-storage scene:

```
C
#include <stdio.h>

void storeItem(int racks, int slotsPerRack,
              int inventory[racks][slotsPerRack],
              int rack, int slot, int itemCode)
{
    if (rack >= 0 && rack < racks &&
        slot >= 0 && slot < slotsPerRack)
        inventory[rack][slot] = itemCode;
}

void printInventory(int racks, int slotsPerRack,
                  int inventory[racks][slotsPerRack])
{
    for (int rack = 0; rack < racks; rack++) {
        printf("Rack %d: ", rack);
        for (int slot = 0; slot < slotsPerRack; slot++)
            printf("%4d", inventory[rack][slot]);
        putchar('\n');
    }
}

int main(void)
{
    int inventory[3][4] = {0};

    storeItem(3, 4, inventory, 1, 2, 205);
    printInventory(3, 4, inventory);

    return 0;
}
```

storeItem receives a copied pointer to the first rack. Through it, the function reaches the caller's original inventory and places item 205 in rack 1, slot 2.

CORE IDEA A real 2D array is an array whose elements are arrays. **int **** is a pointer to a pointer to **int**; it does not automatically describe this layout.

SAVE POINT · ACT II REPLAY

Do not rush through the portal. Close the chapter for a moment, make your prediction, and draw the routes before running the code.

```
C
int racks[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};
printf("%d\n", *((racks + 1) + 2));
```

PREDICT · What value is printed? Identify the row-sized step and the int-sized step.

DRAW THE ROUTES · Draw `racks`, `racks + 1`, `*(racks + 1)`, and `*(racks + 1) + 2` on the same 2 x 3 memory picture.

CHANGE THE GAME · Write the final expression using ordinary `[row][column]` notation, then change it to reach 20.

LEAVE THE DUNGEON · Where outside a game would a contiguous rectangular array and row-wise traversal be useful?

You are ready to continue when you can explain the result without using the phrase “because pointers do that.”

Create Storage at Runtime



Stack, heap, allocation, lifetime, ownership, and dynamic inventories

Mission: create exactly the storage the inventory needs - and know when to release it.

7. The Inventory Size We Do Not Know Yet

The next player starts a new game and chooses the "Collector" class.

The quartermaster turns to us with an awkward question:

"How many inventory slots should I build?"

We do not know. The answer arrives only while the program is running:

```
C
int slots;
printf("How many inventory slots? ");
scanf("%d", &slots);
```

Until now, the sizes were written into the source code. The compiler could see `inventory[5]` or `inventory[3][4]` before the game even began. This time the number belongs to the player.

C99 - the 1999 revision of the C language - permits a variable-length array (VLA), whose length is chosen when execution reaches the declaration:

```
C
int inventory[slots];
```

For modest temporary data, that can be useful. But the quartermaster needs more.

The inventory may be large. It may be created inside a helper function. It must remain alive after that function returns. Later, it may need to grow.

A local array cannot satisfy all of those requirements. Its lifetime ends when execution leaves its block, and very large automatic objects can exhaust the limited storage normally used for active function calls.

So the problem has quietly changed. It is no longer merely "How do we access an address?"

Now we must ask:

CORE IDEA How can the game create an object whose size and lifetime are both decided at runtime?

The quartermaster opens a second vault.



A clockwork quartermaster opens exactly the storage Caitlyn needs.

8. Stack, Heap, and Lifetime

Behind the scenes, the game uses memory for different kinds of work.

One area behaves like a temporary workbench. Every active function receives some space for its call information and local automatic objects. When the function returns, that workspace is reclaimed automatically. On mainstream systems, we call this the **stack**, and one function's portion is commonly called a **stack frame**.

Another area behaves like a warehouse. The program can request a block while it runs and keep that block until it deliberately returns it. This dynamically managed pool is commonly called the **heap**.

The C standard speaks more formally about storage duration and object lifetime; it does not command every implementation to build physical structures with these names. But stack and heap are the practical model used by the systems on which most C programs run.

The quartermaster tries to build the inventory inside a helper function:


```
C
int *createInventoryWrong(int slots)
{
    int items[slots];
    return items;    // wrong
}
```

While the function is running, **items** exists in its local workspace. Returning **items** produces an address leading there. Then the function ends, the local array's lifetime ends, and the caller receives directions to a room that no longer belongs to that object.

The address survived as a number. The inventory did not survive as an object.

We need the warehouse:

```
C
int *createInventory(int slots)
{
    int *items = malloc((size_t)slots * sizeof *items);
    return items;
}
```

Typical stack frame	Dynamically allocated storage
createInventory() active	allocated block at 0x9000
items = 0x9000	[?][?][?][?] ...
items dies on return	block remains alive until free()

Figure 6. The pointer variable and the allocated object have independent lifetimes.

There are two separate things here.

items is a local pointer variable. It normally belongs to the function's stack frame and disappears when the function returns.

The allocated block is a different object in dynamic storage. Its lifetime does not end merely because the local pointer disappears.

When the function returns:

```
C
int *inventory = createInventory(slots);
```

the address is copied into the caller's pointer **inventory**. The temporary signpost **items** vanishes; the warehouse block remains; the caller now holds the route to it.

CORE IDEA The lifetime of a pointer variable and the lifetime of the object it points to are separate stories.

When the temporary workbench overflows

The stack is not endless. Every active call normally consumes part of it for return information, saved machine state, parameters, and local automatic objects.

Consider a quest that calls itself forever:

```
C
void endlessQuest(void)
{
    int localInventory[1000];
    endlessQuest();
}
```

Each call remains active while it waits for the next call to finish. Frame after frame accumulates until the reserved call-stack space is exhausted. A huge local array can consume that space quickly as well.

The usual result is a crash or a report called a **stack overflow**.

Java is not immune. Runaway recursion commonly produces **StackOverflowError**. It may appear less often in beginner Java programs because arrays and ordinary objects are normally created on the heap, while Java reports stack exhaustion explicitly. Its method calls still use a call stack.

Moving large runtime-sized objects to dynamic storage avoids consuming the call stack with those objects. It does not create infinite memory. The warehouse can also run out, and that possibility is why allocation functions can return **NULL**.

9. malloc: Ask the Quartermaster for a Block

The allocation functions are declared in **<stdlib.h>**:

```
C
#include <stdlib.h>
```

Caitlyn asks for **slots** integers:

```
C
int *inventory = malloc((size_t)slots * sizeof *inventory);
```

Read the request from the inside outward.

***inventory** is one **int**, so **sizeof *inventory** is the number of bytes required for one slot. Multiplying by **slots** gives the total number of bytes needed for the inventory. **malloc** reserves one contiguous block of at least that size and returns its starting address.

If the warehouse cannot satisfy the request, it returns a null pointer instead.

That means the next line must not be `inventory[0] = 42`. First we check whether a route was actually returned:

```
C
if (inventory == NULL) {
    fprintf(stderr, "Could not create inventory\n");
    return 1;
}
```

Only then do we enter the block:

```
C
for (int i = 0; i < slots; i++)
    inventory[i] = -1;
```

The indexing looks exactly like array indexing because the pointer leads to the first element of one contiguous sequence. Yet `inventory` is a pointer variable, not an array declaration. The array-like storage was created by `malloc` while the game was running.

The bytes returned by `malloc` begin with indeterminate values. They are not automatically zero, `-1`, or anything meaningful to our game. Until we initialize an element, reading it as though it contained a valid item code is a mistake.

In C, we normally do not cast the result:

```
C
int *inventory = malloc((size_t)slots * sizeof *inventory);
```

`malloc` returns `void *`, and C defines the conversion to an object-pointer type. A cast adds noise and, on older toolchains, can hide the error of forgetting `<stdlib.h>`. C++ follows different conversion rules; this journey is through C.

MAIN QUEST: One Complete Dynamic-Array Program

Before entering the safety door, Caitlyn completes the ordinary journey once: read a size, validate it, allocate, check, use, print, and free.

```

C
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int slots;

    printf("How many inventory slots? ");
    if (scanf("%d", &slots) != 1 || slots <= 0) {
        fprintf(stderr, "Invalid size\n");
        return 1;
    }

    int *inventory = malloc((size_t)slots * sizeof *inventory);
    if (inventory == NULL) {
        fprintf(stderr, "Allocation failed\n");
        return 1;
    }

    for (int i = 0; i < slots; i++)
        inventory[i] = (i + 1) * 10;

    for (int i = 0; i < slots; i++)
        printf("%d ", inventory[i]);
    putchar('\n');

    free(inventory);
    inventory = NULL;
    return 0;
}

```

`size_t` is C's unsigned integer type for object sizes and the value produced by `sizeof`. Here `slots` has already been checked to be positive; the cast converts that count to the type used for sizes. On the first reading, the essential path is still allocation -> NULL check -> use -> free.

BOSS DOOR (OPTIONAL): When the Size Calculation Itself Can Fail

This door is not required for understanding ordinary malloc calls. It handles hostile or extremely large sizes, where even the byte-count multiplication can overflow before malloc receives it.

Suppose `slots` comes from input. Before the quartermaster sees the request, the multiplication itself must be valid.

```

C
#include <stdint.h>
#include <stdlib.h>

if (slots <= 0 || (size_t)slots > SIZE_MAX / sizeof *inventory) {
    fprintf(stderr, "Invalid inventory size\n");
    return 1;
}

```

Without that division check, an enormous multiplication may wrap around and produce a much smaller byte count. `malloc` could successfully return a block that is too small, while the later loop still believes it owns `slots` integers. The allocation would succeed and the program would walk out of bounds - a particularly mischievous failure.

10. **calloc**: Ask for Empty Shelves

The next inventory is a counter board. Every slot should begin at zero.

We could use **malloc** and write zero into each element. Or we can ask the allocator for a cleared block:

```
C
int *inventory = calloc((size_t)slots, sizeof *inventory);
```

Unlike **malloc**, **calloc** receives the number of elements and the size of each element as separate arguments. It allocates one contiguous block and initializes all its bytes to zero.

For an integer array on ordinary C systems, the counters therefore begin at zero:

```
C
printf("%d\n", inventory[0]);
```

But zeroed bytes are not the same thing as a meaningful game state.

If item code 0 means "empty," **calloc** is convenient. If empty slots are represented by **-1**, every element still needs to be written. If we are about to load item codes into every slot from a save file, clearing them first may accomplish nothing useful.

So **calloc** is not the universally safer older sibling of **malloc**. It solves a particular initialization problem.

Function	What it does	Typical game use
malloc	One uninitialized block	You will write every element yourself
calloc	One zeroed block	Zero is a useful initial state
realloc	Resize an existing block	Capacity changes while preserving a prefix
free	End an allocation's lifetime	The block is no longer needed

Table 2. The allocation family: four related jobs, not interchangeable spells.

The choice becomes simple once the scene is clear: use **calloc** when an all-zero initial representation is actually what the program needs; use **malloc** when the program will establish a different state itself.

11. **realloc**: The Backpack Upgrade

Halfway through the dungeon, Caitlyn finds a backpack upgrade.

The old inventory contains **slots** integers. The new one must contain **newSlots**. We could allocate a second block, copy the old items, free the first block, and update the pointer.

realloc performs that resizing operation for us:

```
C
int *resized = realloc(inventory,
    (size_t)newSlots * sizeof *inventory);
```

The allocator now has choices. If space after the existing block is available, it may enlarge the block in place. It may shrink it in place. It may reserve a new block elsewhere, preserve the required old contents, release the old block, and return a different address.

Or it may fail.

That last possibility makes this tempting line dangerous:

```
C
inventory = realloc(inventory, newSize); // risky
```

If the request fails, **realloc** returns **NULL**. The assignment overwrites the only pointer to the original block. The block is still alive, but the game has lost every route to it. It can no longer use or free it.

The inventory has become an unreachable room: a memory leak.

Keep the old route safe until the new one is confirmed:

```
C
int *resized = realloc(inventory,
                      (size_t)newSlots * sizeof *inventory);

if (resized == NULL) {
    fprintf(stderr, "Could not expand inventory\n");
    /* inventory is still valid here */
} else {
    inventory = resized;

    for (int i = slots; i < newSlots; i++)
        inventory[i] = -1;

    slots = newSlots;
}
```

The old elements remain preserved up to the smaller of the old and new sizes. Newly added bytes are not initialized, which is why the new slots are filled explicitly.

There is another consequence. Suppose the targeting UI remembered one slot:

```
C
int *selectedSlot = &inventory[2];
```

If **realloc** moves the block, **selectedSlot** still contains an address inside the old, released block. It has become invalid. After a successful resize, recompute such interior pointers from the new base address.

The backpack may look like the same inventory to the player. In memory, the entire corridor may have moved.

12. free: Close the Vault

Eventually the player sells the inventory, leaves the dungeon, or starts a new game. The dynamic block is no longer needed.

```
C
free(inventory);
inventory = NULL;
```

free does not destroy the pointer variable. It ends the lifetime of the allocated object and returns that block to the allocator.

Immediately after **free(inventory)**, the pointer may still contain the same numerical address. But the program no longer has the right to use the old inventory there. A pointer that still remembers the address of an object whose lifetime has ended is called **dangling**.

Setting the pointer to **NULL** removes the stale route from that particular variable:

```
C
free(inventory);
inventory = NULL;
```

It cannot find and repair every copy of the address:

```
C
int *selectedSlot = &inventory[2];

free(inventory);
inventory = NULL;

printf("%d\n", *selectedSlot); // use-after-free
```

selectedSlot still points into the old block. The main signpost was cleared; the secret side entrance still leads to a room that is no longer ours.

Calling **free(NULL)** is safe and does nothing. Freeing the same allocation twice is not safe:

```
C
free(inventory);
free(inventory); // undefined behaviour
```

The allocator may detect the double free and stop the program. It may have its bookkeeping corrupted and fail later. It may appear to tolerate the mistake. "It did not crash" is not the same as "the operation was valid."

The cleanest way to prevent this confusion is to decide who owns each allocation.

OWNERSHIP RULE Every successful allocation needs one clear owner responsible for releasing it exactly once.

Other pointers may temporarily borrow access. Borrowers must stop using the object before the owner frees it. Ownership is not stored magically inside the pointer value; it is a rule created by the program's design.

13. Returning from a Room That Vanished

Caitlyn finds a treasure worth 50 coins. A helper function tries to create it:

```
C
int *createTreasure(void)
{
    int treasure = 50;
    return &treasure; // wrong
}
```

The returned address is genuine while **createTreasure** is still running. It leads to the local integer **treasure**.

Then the function returns. The object's lifetime ends. The address reaches a place where the treasure used to exist.

This is the same mistake as the local inventory, now reduced to one coin chest.

Dynamic storage changes the lifetime:

```
C
int *createTreasure(void)
{
    int *treasure = malloc(sizeof *treasure);

    if (treasure != NULL)
        *treasure = 50;

    return treasure;
}
```

The local pointer disappears on return, but the allocated integer remains alive. The caller receives the route and, with it, ownership:

```
C
int *treasure = createTreasure();

if (treasure != NULL) {
    printf("Treasure value: %d\n", *treasure);
    free(treasure);
    treasure = NULL;
}
```

Names such as **createInventory**, **allocateBuffer**, or **cloneHero** hint that a new resource is being handed to the caller. Good documentation should say the ownership rule explicitly.

Not every returned pointer owns dynamic memory. A function may return a pointer to caller-owned storage or to an object with static storage duration. Such pointers can be valid, but the caller must not automatically **free** them.

The pointer tells us where an object is. It does not, by itself, tell us who owns the object or how long it will remain alive.

14. Three Ways to Build a Dynamic 2D Inventory

The collector class has now become ridiculous.

One line of slots is no longer enough. The inventory needs **racks** rows, each containing **slotsPerRack** positions. Both numbers are chosen while the game is running.

The player still wants to write:

```
C
inventory[rack][slot]
```

But before choosing a pointer declaration, we must choose what will actually exist in memory. Pointer syntax follows the layout; it does not invent the layout for us.

The quartermaster offers three construction plans.

Plan A: one long floor of slots

The simplest plan is one contiguous block of integers:

```
C
int *inventory = malloc((size_t)racks * slotsPerRack
    * sizeof *inventory);
```

The block contains rack 0, followed by rack 1, followed by rack 2. But the type is only `int *`, so C sees one long sequence of integers. It does not remember where one logical rack ends.

We calculate the position ourselves:

```
C
inventory[rack * slotsPerRack + slot] = itemCode;
```

This plan is compact, contiguous, cache-friendly, and released with one `free(inventory)`. The cost is that the flattening formula remains visible wherever two-dimensional indexing is needed.

Plan B: teach the pointer the rack width

The second plan uses the same kind of contiguous block. The difference is the pointer's destination type:

```
C
int (*inventory)[slotsPerRack] =
    malloc((size_t)racks * sizeof *inventory);
```

Stop at the declaration and read it from the name outward.

`inventory` is a pointer. Dereferencing it produces an array of `slotsPerRack` integers. That array is one complete rack.

So one pointer step now means one rack:

```
C
inventory + 1    // next complete rack
inventory[rack] // one rack
inventory[rack][slot] // one slot in that rack
```

`sizeof *inventory` is the size of one full rack, not the size of one pointer and not the size of one integer. Multiplying it by `racks` requests exactly enough bytes for every rack.

This declaration does **not** create **slotsPerRack** pointers. There is one pointer variable named **inventory**. The allocated block contains only adjacent integers arranged into adjacent rack-sized groups.

TEXT

inventory ----> [rack 0][rack 1][rack 2] ...
one pointer each rack contains slotsPerRack integers

One allocation means one later **free(inventory)**.

The complete pointer-to-rack game

First, the helper functions. Their array parameters are adjusted to pointers to runtime-sized racks:

```
C
#include <stdio.h>
#include <stdlib.h>

void fillInventory(int racks, int slotsPerRack,
                  int inventory[racks][slotsPerRack])
{
    for (int rack = 0; rack < racks; rack++)
        for (int slot = 0; slot < slotsPerRack; slot++)
            inventory[rack][slot] = -1;
}

void printInventory(int racks, int slotsPerRack,
                   int inventory[racks][slotsPerRack])
{
    for (int rack = 0; rack < racks; rack++) {
        printf("Rack %d: ", rack);
        for (int slot = 0; slot < slotsPerRack; slot++)
            printf("%4d", inventory[rack][slot]);
        putchar('\n');
    }
}
```

Now **main** learns both dimensions, creates the matching pointer type, and asks for **racks** complete rack objects:

Complete program (continued): main

```
C
int main(void)
{
    int racks, slotsPerRack;

    printf("Enter racks and slots per rack: ");
    if (scanf("%d%d", &racks, &slotsPerRack) != 2 ||
        racks <= 0 || slotsPerRack <= 0) {
        fprintf(stderr, "Invalid dimensions\n");
        return 1;
    }

    int (*inventory)[slotsPerRack] =
        malloc((size_t)racks * sizeof *inventory);

    if (inventory == NULL) {
        fprintf(stderr, "Could not allocate the inventory\n");
        return 1;
    }

    fillInventory(racks, slotsPerRack, inventory);

    inventory[0][0] = 101; // sword
    if (racks > 1 && slotsPerRack > 2)
        inventory[1][2] = 205; // shield

    printInventory(racks, slotsPerRack, inventory);

    free(inventory);
    inventory = NULL;
    return 0;
}
```

Watch the route travel through the program.

In **main**, **inventory** points to a runtime-sized rack. **malloc** creates a contiguous sequence of **racks** such racks. Passing **inventory** copies the address of the first rack into each helper. Because **slotsPerRack** is known there, the compiler knows the size of one pointer step, and ordinary **inventory[rack][slot]** notation works.

For a genuinely rectangular inventory, this is often the most expressive construction: one allocation, natural indexing, and contiguous storage.

SIDE QUEST: Plan C - Build Each Rack Separately

The third plan changes the building itself.

First, create an array of rack pointers. Then create each rack as a separate allocation:

```

C
int **inventory = malloc((size_t)racks * sizeof *inventory);

if (inventory == NULL)
    return NULL;

for (int rack = 0; rack < racks; rack++) {
    inventory[rack] =
        malloc((size_t)slotsPerRack * sizeof *inventory[rack]);

    if (inventory[rack] == NULL) {
        while (rack > 0)
            free(inventory[--rack]);

        free(inventory);
        return NULL;
    }
}

```

Now `int **` is not pretending to be a 2D array. The first allocation genuinely contains `int *` objects, and each of those pointers genuinely leads to a separate integer array.

```

TEXT
inventory -> [rack pointer 0] ----> [int int int int]
           [rack pointer 1] ----> [int int int int]
           [rack pointer 2] ----> [int int int int]

```

The two indexing operations follow two real pointer levels:

```

C
inventory[rack]    // obtain one rack pointer
inventory[rack][slot] // follow it to one integer

```

The racks may live at unrelated addresses. That is why this layout cannot be passed to a function expecting `int inventory[][slotsPerRack]`. That function expects adjacent rack objects. Here, the first block contains addresses instead.

Construction required several allocations, so destruction must mirror it:

```

C
for (int rack = 0; rack < racks; rack++)
    free(inventory[rack]);

free(inventory);
inventory = NULL;

```

The extra work earns flexibility. Each rack may have a different length, be replaced independently, or live separately for another design reason.

The complete separately allocated version

The creation function must clean up even when allocation fails halfway through:

```

C
#include <stdio.h>
#include <stdlib.h>

```

```

int **createInventory(int racks, int slotsPerRack)
{
    int **inventory = malloc((size_t)racks * sizeof *inventory);

    if (inventory == NULL)
        return NULL;

    for (int rack = 0; rack < racks; rack++) {
        inventory[rack] =
            calloc((size_t)slotsPerRack, sizeof *inventory[rack]);

        if (inventory[rack] == NULL) {
            while (rack > 0)
                free(inventory[--rack]);
            free(inventory);
            return NULL;
        }
    }

    return inventory;
}

void destroyInventory(int **inventory, int racks)
{
    for (int rack = 0; rack < racks; rack++)
        free(inventory[rack]);
    free(inventory);
}

```

The caller receives the address of the pointer array and later returns the entire construction to the matching destructor:

Complete program (continued): caller

```

c
int main(void)
{
    int racks = 3;
    int slotsPerRack = 4;
    int **inventory = createInventory(racks, slotsPerRack);

    if (inventory == NULL) {
        fprintf(stderr, "Could not allocate the inventory\n");
        return 1;
    }

    inventory[1][2] = 205;
    printf("Item code: %d\n", inventory[1][2]);

    destroyInventory(inventory, racks);
    inventory = NULL;
    return 0;
}

```

There were **racks + 1** successful allocations - one pointer array and one integer array per rack. Cleanup releases every rack before releasing the pointer array that holds their addresses.

Layout	Base type	Trade-off
Flat block	int *	One allocation; contiguous; manual rack * slotsPerRack + slot indexing
Pointer to runtime-sized rack	int (*)[slotsPerRack]	One allocation; contiguous; natural inventory[rack][slot] indexing
Rack pointers	int **	Many allocations; rack capacities may vary; more cleanup and indirection

Table 3. *The type follows the chosen memory layout.*

The quartermaster's final question is the right one:

CORE IDEA "What shape must exist in memory?"

Once that is answered, the pointer type stops looking like a spell. It becomes a description.

SAVE POINT · ACT III REPLAY

Do not rush through the portal. Close the chapter for a moment, make your prediction, and draw the routes before running the code.

```
C
int *resized = realloc(inventory, newBytes);

if (resized == NULL)
    printf("%d\n", inventory[0]);
```

PREDICT · Assume realloc returns NULL. Is inventory[0] still legal to read? What would change if realloc had been assigned directly to inventory?

DRAW THE ROUTES · Draw the local pointer returned by createInventory separately from the dynamically allocated block. Cross out only the object whose lifetime actually ends when the function returns.

CHANGE THE GAME · Change the complete malloc program so all elements begin at zero with calloc. Which initialization loop can disappear, and when would removing it be wrong?

LEAVE THE DUNGEON · Name one real program whose required storage size is known only while it runs.

You are ready to continue when you can explain the result without using the phrase “because pointers do that.”

Change the Route Itself



Pointers to pointers, changing targets, and output parameters

Mission: let a function change not only an object, but which object the caller can reach.

15. The Targeting Problem: When One Pointer Is Not Enough



Caitlyn changes which monster the targeting system selects.

Back in the arena, a slime waits on the left and a dragon waits on the right.

```
C
typedef struct {
    const char *name;
    int health;
} Enemy;

Enemy slime = {"Slime", 30};
Enemy dragon = {"Dragon", 300};

Enemy *currentTarget = &slime;
```

The targeting reticle currently points to the slime.

If a function only needs to damage whichever enemy is selected, a simple pointer is enough:

```
C
void damageTarget(Enemy *target, int damage)
{
    target->health -= damage;
}
```

The function receives a copied address, follows it, and changes the **Enemy** at the destination. We have done this before with health.

Then the dragon roars, and the game must move the reticle.

The first attempt looks reasonable:

```
C
void selectTarget(Enemy *target, Enemy *newTarget)
{
    target = newTarget;
}
```

The function changes **target** from the slime's address to the dragon's address. Inside the function, the reticle appears to move.

When the function returns, **currentTarget** in the caller still points to the slime.

Why?

Because **target** was only a local copy of the caller's pointer. Changing ***target** would change the selected enemy object. Changing **target** changes only the copied route.

The problem is no longer "How can the function reach the enemy?" It already can.

The new problem is:

CORE IDEA How can the function reach and modify the caller's pointer variable itself?

We solve it exactly as we solved the original health problem. To change a caller-owned object, pass that object's address. This time the caller-owned object happens to be a pointer.

```
C
void selectTarget(Enemy **targetSlot, Enemy *newTarget)
{
    *targetSlot = newTarget;
}
```

The call supplies the address of **currentTarget**:

```
C
selectTarget(&currentTarget, &dragon);
```

targetSlot		currentTarget		dragon
stores ¤tTarget	->	stores &dragon	->	Enemy object health = 300

Figure 7. The parameter reaches the caller's pointer, which reaches the selected enemy.

Read the declaration from the variable name outward:

CORE IDEA **targetSlot** is a pointer to an **Enemy ***.

Then follow the levels:

targetSlot is the address of the caller's pointer variable.

***targetSlot** is the caller's pointer **currentTarget**.

****targetSlot** is the **Enemy** object reached through **currentTarget**.

The assignment:

```
C
*targetSlot = newTarget;
```

does not modify an enemy. It replaces the address stored in the caller's pointer. The caller's route now leads to the dragon.

CORE IDEA A pointer lets a function change my object. A pointer to my pointer lets a function change what I point to.

When double pointers appear without dragons

The targeting reticle is dramatic, but the pattern is ordinary. A pointer-to-pointer parameter appears whenever a function must replace a pointer owned by its caller.

For example, a function may allocate an inventory while using its direct return value as a success flag:

```
C
int createInventory(int **outInventory, int slots)
{
    int *items = malloc((size_t)slots * sizeof *items);

    if (items == NULL)
        return 0;

    *outInventory = items;
    return 1;
}
```

The caller passes the address of its **int ***:

```
C
int *inventory = NULL;

if (!createInventory(&inventory, 20))
    fprintf(stderr, "Allocation failed\n");
```

Returning the allocated pointer directly is often simpler. The output-parameter form earns its place when the ordinary return value already reports status or when the function must produce several results.

You may sometimes see the general pattern written as **T ****. Here **T** is not C syntax by itself; it is a placeholder meaning "some object type."

```
C
int **p;    // pointer to pointer to int
char **p;   // pointer to pointer to char
Enemy **p;  // pointer to pointer to Enemy
```

Writing only ****p** is not a declaration. It is an expression that dereferences an already-declared pointer twice.

BOSS DOOR (OPTIONAL): Is There Ever a Reason for ***?

The next door is optional.

Suppose the caller owns a **char **dialogue**: a pointer to an array of string pointers. A function must allocate that whole pointer array and replace the caller's **char ****.

The function therefore needs the address of a **char ****. That address has type **char *****:

```
C
int createDialogue(char ***outLines, int count)
{
    char **lines = calloc((size_t)count, sizeof *lines);

    if (lines == NULL)
        return 0;

    *outLines = lines;
    return 1;
}
```

Important: this calloc creates only count slots for char * pointers, initially null. It has not created any character arrays and it has not copied any dialogue text. Each string would need its own storage later, and cleanup would free those strings before freeing the outer pointer array.

The caller uses:

```
C
char **dialogue = NULL;
createDialogue(&dialogue, 5);
```

The third `*` does not mean "three-dimensional array." It means that the function must modify a caller-owned variable whose type already contains two pointer levels.

Every extra star should answer a real question: what object are we trying to reach, and how many stored routes stand between us and it?

SAVE POINT · ACT IV REPLAY

Do not rush through the portal. Close the chapter for a moment, make your prediction, and draw the routes before running the code.

```
C
void selectTarget(Enemy *target, Enemy *newTarget)
{
    target = newTarget;
}

selectTarget(currentTarget, &dragon);
printf("%s\n", currentTarget->name);
```

PREDICT · If `currentTarget` initially points to the slime, which name is printed? Why does the reticle fail to move?

DRAW THE ROUTES · Draw `targetSlot` -> `currentTarget` -> slime before the correct call, then redraw it after `*targetSlot` receives `&dragon`.

CHANGE THE GAME · Modify `createInventory(int **outInventory, int slots)` so the caller frees the allocation only after a successful return.

LEAVE THE DUNGEON · Find a non-game function that might need to replace a caller-owned buffer or cursor.

You are ready to continue when you can explain the result without using the phrase “because pointers do that.”

Survive the Dangerous Parts



Bounds, lifetime, cleanup, ownership, and debugging tools

Mission: keep the power of pointers without setting fire to the memory map.

16. The Gallery of Pointer Disasters

The final dungeon does not introduce a new star. It tests whether every route we already created is still valid.

At the first door, an uninitialized pointer contains no destination chosen by the program. Dereferencing it follows nonsense.

At the second, a null pointer deliberately points nowhere, but the code dereferences it anyway.

At the third, a pointer begins inside a valid array and then walks beyond the final element.

At the fourth, the destination once existed but its lifetime has ended. The route dangles.

At the fifth, every route to a live allocation has been lost. The room remains occupied forever, but nobody can enter it or return it to the allocator.

At the sixth, two pieces of code both believe they own the same allocation and both call **free**.

These failures look different on the screen, but most are questions of four things: validity, bounds, lifetime, and ownership.

Bug	What went wrong	Design habit
Uninitialized pointer	Dereference before assigning a valid address	Initialize from an object, allocation, or NULL
Null dereference	Dereference NULL	Test before use when NULL is possible
Out-of-bounds access	Move outside an array or allocation	Carry and check element counts
Dangling pointer	Object's lifetime ended	Track lifetime; invalidate non-owning aliases
Use-after-free	Dereference after free	Stop access before freeing; clear ownership
Memory leak	Lose every pointer to a live allocation	Define one owner and cleanup paths
Double free	Free the same allocation twice	One owner; one successful free
Wrong free	Free a local object, interior pointer, or foreign pointer	Free only the exact address returned by allocation

Table 4. Pointer failures are usually failures of validity, bounds, lifetime, or ownership.

The phrase nobody likes: undefined behaviour

Suppose the hero steps beyond the inventory, follows a dangling pointer, or frees the same block twice. What error will C print?

There is no required answer.

When the C standard calls an operation **undefined behaviour**, it places no requirements on what follows. The program may crash immediately. It may print plausible nonsense. It may corrupt another object and fail ten minutes later. It may appear to work during every practice run and betray us during the demonstration.

That last outcome is why "it worked on my machine" proves only that one execution did not reveal the bug.

C's compiler often cannot determine whether an address still names a live object, whether a caller owns it, or whether a computed position remains inside the correct allocation. The language gives the programmer control and expects the program's design to preserve those truths.

Give the game suspicious companions

We do not have to investigate alone. Compile with strong warnings and runtime sanitizers during development:

```
BASH
gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion \
-g -fsanitize=address,undefined game.c -o game
```

AddressSanitizer catches many out-of-bounds accesses, use-after-free errors, and double frees. UndefinedBehaviorSanitizer catches several other invalid operations. Compiler warnings expose suspicious declarations and conversions before the game runs.

On Linux, Valgrind can also report leaks and invalid memory access, although sanitizers are often faster for everyday development.

These tools do not create an ownership model for us. They are excellent at shouting when the model we believed in was false.

17. The Five Questions at the Last Gate

At the dungeon exit, a gatekeeper examines every pointer carried by the program.

The gate opens only after five questions.

1. **What is the pointer's type?** What kind of object should exist at the destination, and how large is one step?
2. **Where does it point now?** To a live object, to the one-past boundary of an array, or to no object at all?

-
3. **How long will that object remain alive?** Until a block ends, until a function returns, for the whole program, or until its owner calls **free**?
 4. **Who owns the object?** Who must release it, and who is merely borrowing access?
 5. **What are the valid bounds?** How many elements can legally be reached from this starting address?

Most pointer bugs are one of these questions answered incorrectly - or never asked.

The checklist is not separate theory added after the adventure. It is the condensed memory of everything that went wrong along the way.

SAVE POINT · ACT V REPLAY

Do not rush through the portal. Close the chapter for a moment, make your prediction, and draw the routes before running the code.

```
C
int *alias = inventory;
free(inventory);
inventory = NULL;
printf("%d\n", *alias);
```

PREDICT · Is any particular output guaranteed? Explain why clearing inventory does not repair alias.

DRAW THE ROUTES · Draw one owner and two borrowed aliases. Mark the exact moment after which every borrowed route becomes invalid.

CHANGE THE GAME · Repair a double-free example by assigning NULL after the first free, then explain why this protects only that one pointer variable.

LEAVE THE DUNGEON · Name one situation where bounds, lifetime, or ownership would be more important than the pointer syntax itself.

You are ready to continue when you can explain the result without using the phrase “because pointers do that.”

18. The Whole Point

At the beginning of the dungeon, we had one ordinary integer.

The monster attacked, a function changed its private copy, and the hero remained at 100 health. That small failure gave us a reason to ask for the original object's address.

The address became a second route to the same integer.

Then the inventory placed several integers side by side, and the route learned to move from one element to the next. One-past became a finish line. A pointer to a whole rack made one step cross an entire row.

The player chose an inventory size at runtime, and local storage could no longer give us the lifetime we needed. Dynamic allocation created the object after the program began. **realloc** allowed it to move and grow. **free** ended its lifetime. Ownership decided who had the right - and the duty - to do that.

The targeting reticle presented one final twist. Changing an enemy through a pointer was easy. Changing the caller's pointer required a route to the pointer itself.

Across every room, the same pattern held: first identify the object that must change, then decide which route the program must carry to reach it safely.

Nothing in this journey was really about admiring the asterisk.

CORE IDEA Pointers separate an object from the route used to reach it. They let different parts of a program share an object, move through related objects, choose lifetimes at runtime, replace routes, and connect separately created objects into changing structures.

That is the power C gives us.

The invoice arrives with it: bounds, lifetime, ownership, and cleanup are now our responsibility.

The compiler will faithfully follow the addresses we provide. It is worth making sure each one still leads somewhere real.

Save Point Quick Checks

ACT I • The program prints 100 and 0. The parameter `p` is only a copied pointer; target remains unchanged.

ACT II • The expression prints 60. The first `+1` crosses one complete three-int row; the later `+2` crosses two int elements.

ACT III • When `realloc` fails through a temporary pointer, the original allocation and inventory remain valid.

ACT IV • The incorrect one-pointer function prints Slime. Only its local copy of the route changed.

ACT V • No output is guaranteed. `alias` became dangling when `free` ended the allocation's lifetime.

Compact Reference: The Map Caitlyn Carries Home

```
C
int x = 10;      // one int object
int *p = &x;     // p stores the address of x

*p              // the int reached through p
&x             // the address of x
&p             // the address of the pointer p

p + 1           // next int position within the same array
p[i]           // the same as *(p + i)

int **q = &p;    // q points to the pointer p
*q            // p
**q           // x

int *a = malloc(n * sizeof *a);
int *b = calloc(n, sizeof *b);

int *tmp = realloc(a, newN * sizeof *a);
if (tmp != NULL)
    a = tmp;

free(a);
a = NULL;
```

If these lines now read like a map rather than an incantation, Caitlyn is ready for the next dungeon.